

Switch It

Direct driver-to-driver parking handoff

A web MVP that coordinates one short street-parking exchange between a leaving driver and a specific seeker.

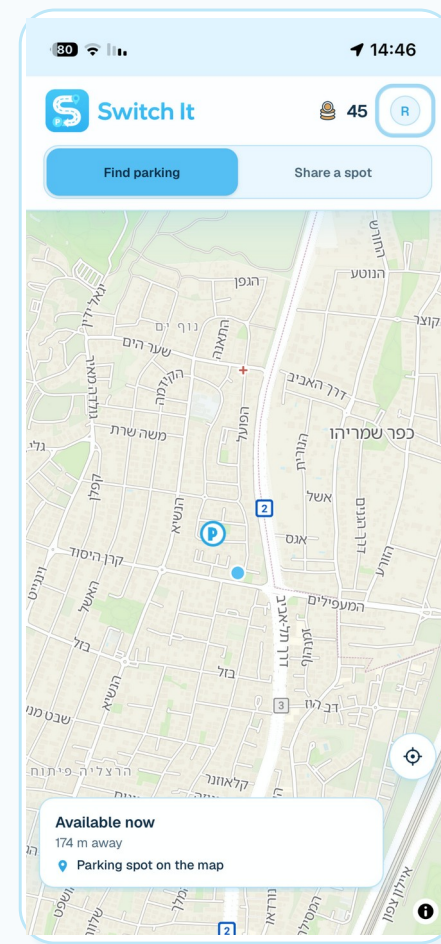
RUNI Internet Technologies · Final submission

App URL

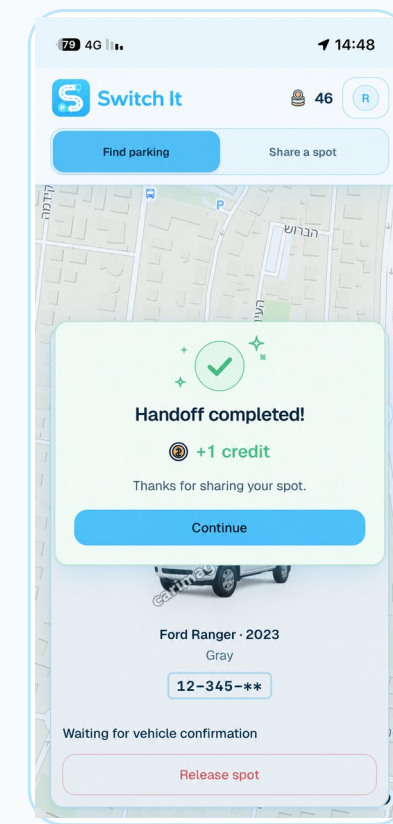
<https://switch-it-wine.vercel.app>

Repository

github.com/rotemkap10/switch-it



Find parking



Completion

What I built

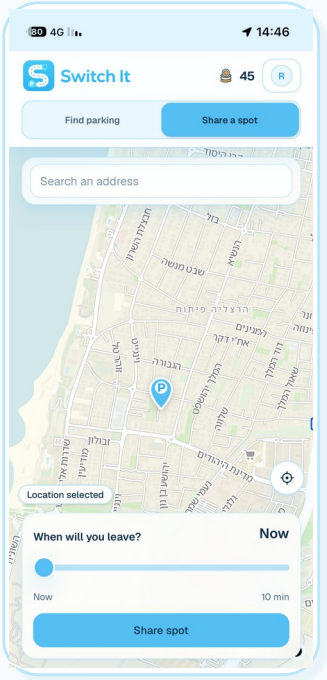
A focused product MVP: one parking opportunity, one seeker, one confirmed handoff.

The problem

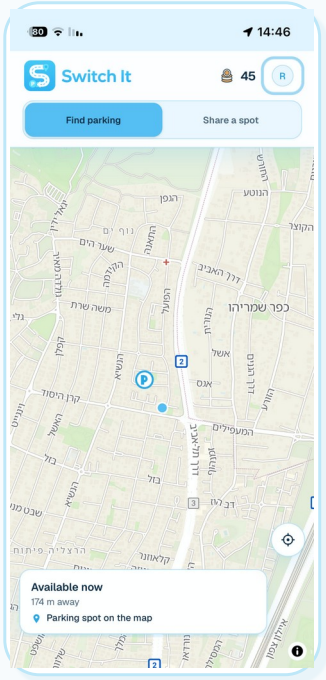
- Street parking is mostly uncertainty.
- Drivers circle while spots free up without coordination.
- A report is not the same as a handoff.

The product

- Publisher shares a spot that will be available now or soon.
- Seeker claims a specific spot and navigates to it.
- Credits reward reliable sharing and keep accountability.



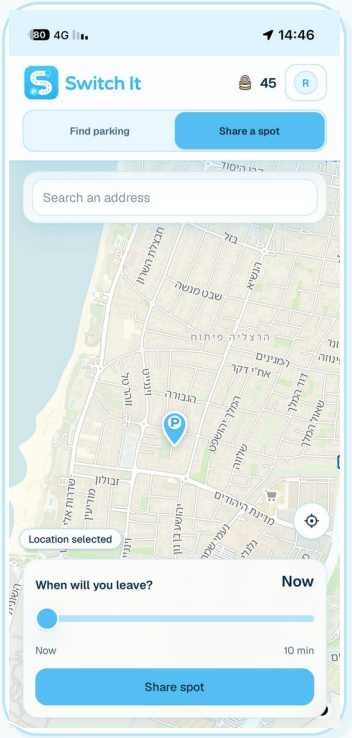
Publisher



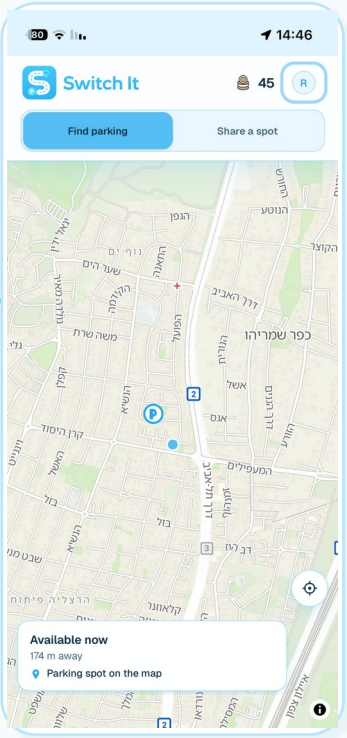
Seeker

How to use it

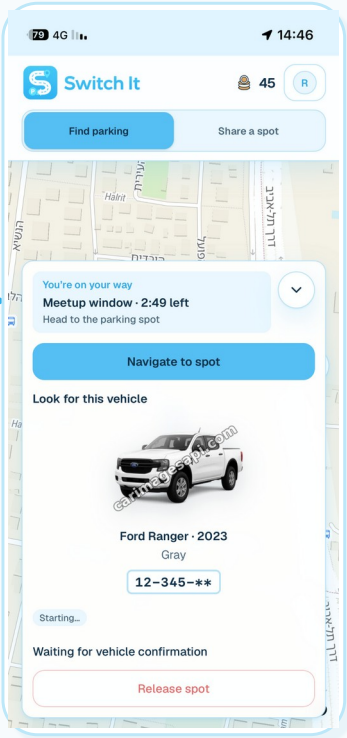
The full flow the tester should try with two demo users.



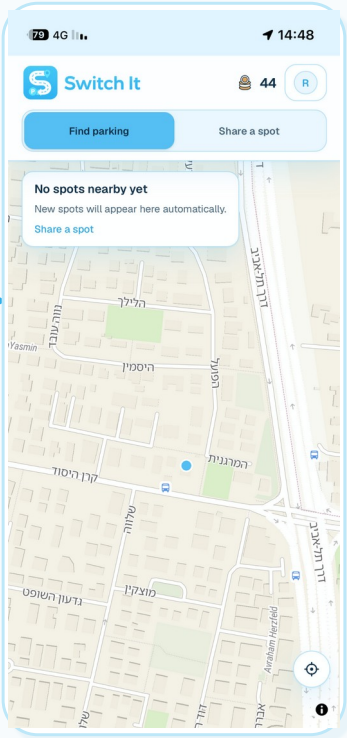
Publish spot



Find & claim



Navigate / handoff



Complete

Tester flow

- 1 Publisher posts a spot
- 2 Seeker claims it
- 3 Seeker navigates to the spot
- 4 Both see the live handoff window
- 5 Publisher verifies last 2 plate digits
- 6 Credits and history update

Credits move only after verified completion, not when a claim is created.

Technical snapshot

Built as a real deployed full-stack web app, with database rules for the critical flows.

Next.js + TypeScript

App Router UI, Server Actions, typed forms

Supabase Auth

Email confirmation, password recovery, authenticated sessions

PostgreSQL + RPC

Atomic claims, RLS, timing and credit transactions

Realtime + Edge Function

Live handoff updates and web/native location publishing

MapLibre + MapTiler

Interactive maps, address search and current-location camera

Vercel + PWA

Production URL, service worker and stale-client recovery

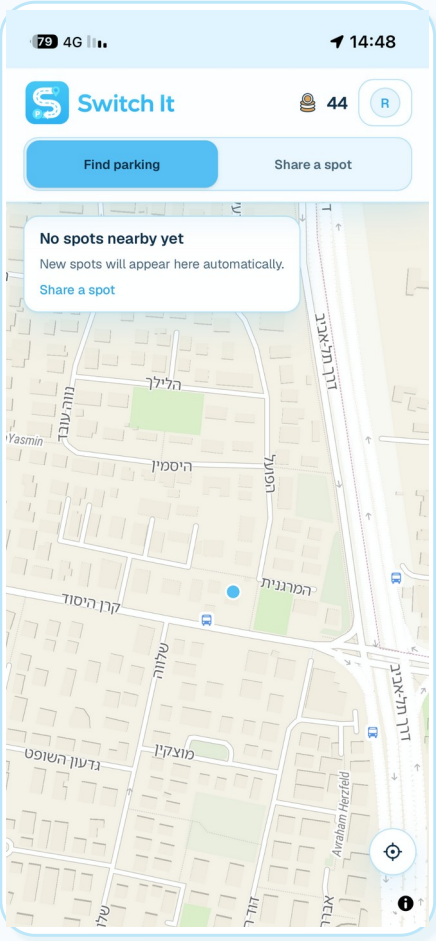
Core database entities: profiles, parking_spots, claims, credit_transactions.

Current test suite: 277 files / 1744 tests passed.

What was easy

The fastest parts were product iteration and using the course stack end-to-end.

Building the basic UI	React components made it easy to move quickly from rough screens to a usable interface.
Shipping a public URL	Vercel made deployment simple and gave the project a real production address.
Supabase basics	Authentication, simple tables and regular CRUD flows were quick to connect.
Using coding agents	AI tools helped generate tests, improve copy, refactor repeated logic and document the system.



Polished completion flow

What was difficult

The challenging parts were the real-world edge cases behind a simple parking flow.

Concurrency

Two seekers may claim at the same time, so the database must choose one winner.

Timing

Future availability, "I'm leaving now", meetup windows and expiry had to stay consistent for both users.

Realtime + location

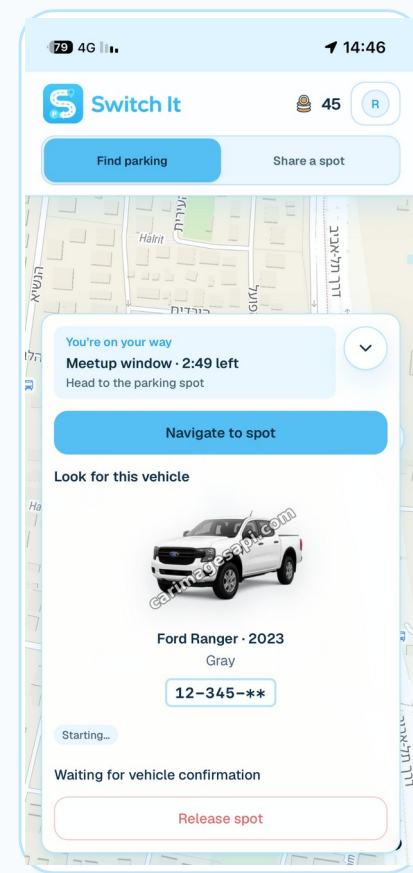
Live handoff state and live location had to work across browser, PWA and mobile differences.

Auth edge cases

Email verification, duplicate-email privacy and password reset flows required careful routing and UX.

Reliability after deploy

Long-lived tabs, PWA cache, stale chunks and Realtime reconnects needed defensive handling.



Active handoff state

The main lesson: the app looks simple, but correctness depends on server-side rules, realtime state and careful edge-case handling.